

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Построение и анализ алгоритмов»
Тема: «Поиск с возвратом»

Студент гр. 4345

Андросов М.А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

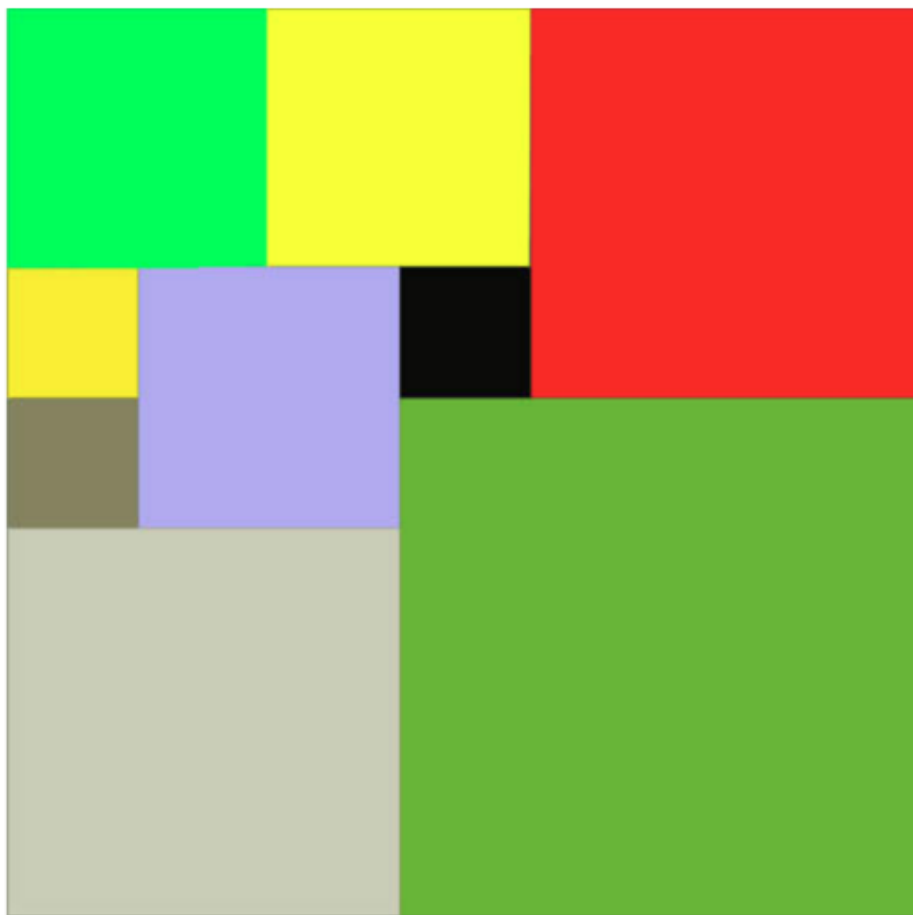
Цель работы

Изучить и реализовать алгоритм поиска с возвратом.

Задание

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 1 до $N - 1$, и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу - квадрат размера N . Он может получить ее, собрав из уже имеющихся обрезков(квадратов).

Например, столешница размера 7×7 может быть построена из 9 обрезков.



Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Входные данные

Размер столешницы – одно целое число N ($2 \leq N \leq 20$).

Выходные данные

Одно число K , задающее минимальное количество обрезков(квадратов), из которых можно построить столешницу(квадрат) заданного размера N . Далее должны идти K строк, каждая из которых должна содержать три целых числа x, y и w , задающие координаты левого верхнего угла ($1 \leq x, y \leq N$) и длину стороны соответствующего обрезка(квадрата).

Пример входных данных

7

Соответствующие выходные данные

9

1 1 2

1 3 2

3 1 1

4 1 1

3 2 2

5 1 3

4 4 4

1 5 3

3 4 1

Вариант 4р.

Рекурсивный бэктрекинг. Расширение задачи на прямоугольные поля, рёбра квадратов меньше рёбер поля. Подсчёт количества вариантов покрытия минимальным числом квадратов.

Выполнение работы

Разработан рекурсивный алгоритм поиска с возвратом(backtracking).

1. Описание алгоритма

На каждом шаге выбирается первая свободная клетка и пробуются все возможные квадраты начиная с наибольшего. Если выбор приводит к ухудшению оптимального решения, дальнейший перебор ветви прекращается. После проверки варианта алгоритм возвращается к предыдущему состоянию и пробует следующий.

2. Способ хранения частичных решений

для хранения состояния задачи и промежуточных результатов используются:

- двумерный массив(bool) field – позволяет проверять доступность места для нового квадрата.
- Динамический массив figures - хранящий текущий набор размещенных квадратов как «тройку» (x,y,w) – координаты левого верхнего угла и длину.
- Динамический массив ans_figures – оптимальный набор размещенных квадратов, покрывающих всю поверхность field.
- Счетчик best – минимальное количество квадратов , необходимых для покрытия поверхности.
- Счетчик ways – Подсчёт количества вариантов покрытия минимальным числом квадратов.

3. Используемые оптимизации алгоритма

1) Жадный выбор:

Квадраты перебираются от наибольшего к наименьшему, что позволяет эффективнее искать оптимальные решения.

2) нижняя граница:

На каждом шаге вычисляется нижняя граница $\{\text{ceil}(\text{remained_cells} / (\text{max_size} * \text{max_size}))\}$, где remained_cells – свободные клетки , max_size –

максимальная сторона квадрата , помещающегося в field. Если количество фигур + данная оценка $\geq$ лучшего случая , то ветвь отсекается.

3) верхняя граница:

Перед запуском основного поиска выполняется жадный алгоритм (get_greedy()) для получения начального значения best.

4. Описание рекурсивной функции

В работе выполнена функция backtrack.

Сигнатура: void backtrack(vector<tuple<int,int,int>>& figures).

Назначение:

Выполняет пошаговое построение покрытия поля.

Аргументы:

figures – ссылка на текущий вектор установленных квадратов(частичное решение).

Описание:

- 1) проверка условия отсечения(size $\geq$ best)
- 2) поиск первой свободной клетки(функция free_cell())
- 3) проверка условия отсечения по нижней границе
- 4) получаем максимально возможный размер квадрата для точки. Если квадрат подходит , то вызывается функция place() - заполнение поля, добавляется в figures.

5. Оценка сложности алгоритма

Временная сложность - $O(b^h)$, где b – кол-во ветвей , а h – глубина рекурсии.

Пространственная сложность - $O(n * m)$

Разработанный программный код см. в приложении А.

Тестирование

Таблица 1 — Результаты тестирования.

№	Входные данные	Выходные данные
1	7 7	ways: 1 min: 9 x y w 1 1 4 1 5 3 4 5 1 4 6 2 5 1 3 5 4 2 6 6 2 7 4 1 7 5 1
2	4 4	ways: 1 min: 4 x y w 1 1 2 1 3 2 3 1 2 3 3 2
3	13 17	ways: 1 min: 8 x y w 1 1 13 1 14 4 5 14 4 9 14 4 13 14 1 13 15 1 13 16 1 13 17 1
4	19 19	ways: 1 min: 13 x y w 1 1 7 1 8 6 1 14 6 7 8 1 7 9 4 7 13 7

			8 1 8
			11 9 4
			14 13 1
			14 14 6
			15 9 5
			16 1 4
			16 5 4

Вывод

В ходе работы был реализован и протестирован рекурсивный алгоритм `backtracking` для покрытия прямоугольного поля квадратами минимальным числом фигур. За счет введенных оптимизаций удалось существенно сократить время работы алгоритма и обеспечить практическую эффективность поиска для n и $m \leq 21$.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: main.cpp

```
#include <iostream>
#include <vector>
#include <tuple>
#include <climits>
#include <cmath>

using namespace std;

int n, m;
vector<vector<bool>> field;
vector<tuple<int,int,int>> ans_figures;
int best = INT_MAX;
long long ways = 0;

pair<int,int> free_cell(const vector<vector<bool>>& board)
{
    for(int i = 0; i < n; i++){
        for(int j = 0; j < m; j++){
            if(!board[i][j])
                return {i, j};
        }
    }
    return {-1, -1};
}

bool fits(const vector<vector<bool>>& board, int x, int y, int size)
{
    if (x + size > n || y + size > m)
        return false;

    for(int i = 0; i < size; i++){
        for(int j = 0; j < size; j++){
            if(board[x + i][y + j])
                return false;
        }
    }
}
```

```

        }
    }
    return true;
}

void place(vector<vector<bool>>& board, int x, int y, int size, bool val)
{
    for(int i = 0; i < size; i++){
        for(int j = 0; j < size; j++){
            board[x + i][y + j] = val;
        }
    }
}

int max_square_size(const vector<vector<bool>>& board, int x, int y)
{
    int mx = min(n - x, m - y);
    if(n == m && mx == n) mx--;
    int res = 0;

    for(int i = 1; i <= mx; i++){
        if(!fits(board, x, y, i))
            break;
        res = i;
    }
    return res;
}

int get_remained(int x, int y)
{
    int c = 0 ;
    for(int i = 0; i < n; i++){
        for(int j = 0; j < m; j++){
            if(!field[i][j]) c++;
        }
    }

    return c;
}

```

```

void backtrack(vector<tuple<int,int,int>>& figures)
{
    if ((int)figures.size() >= best)
        return;

    auto [x, y] = free_cell(field);

    if(x == -1){
        if ((int)figures.size() < best){
            best = (int)figures.size();
            ans_figures = figures;
            ways = 1;
        }
        else if((int)figures.size() == best){
            ways++;
        }
        return;
    }

    int remained_cells = get_remained(x,y);
    int max_size = max_square_size(field, x, y);
    if (ceil(remained_cells / (max_size * max_size)) + figures.size() >=
best) return;

    for(int size = max_size; size >= 1; size--){
        if(fits(field, x, y, size)){
            place(field, x, y, size, true);
            figures.push_back({x + 1, y + 1, size});

            backtrack(figures);

            figures.pop_back();
            place(field, x, y, size, false);
        }
    }
}

```

```

int get_greedy(vector<tuple<int,int,int>>& greedy_figures)
{
    vector<vector<bool>> temp(n, vector<bool>(m, false));
    int c = 0;

    while(true){
        auto [x, y] = free_cell(temp);
        if(x == -1) break;

        int mx = max_square_size(temp, x, y);
        place(temp, x, y, mx, true);
        greedy_figures.push_back({x + 1, y + 1, mx});
        c++;
    }
    ways = 1;

    return c;
}

int main()
{
    cin >> n >> m;
    if(n > m) swap(n, m);

    field.assign(n, vector<bool>(m, false));

    vector<tuple<int,int,int>> greedy_figures;
    best = get_greedy(greedy_figures);
    ans_figures = greedy_figures;

    vector<tuple<int,int,int>> figures;
    figures.reserve(n * m);

    backtrack(figures);

    cout << "ways: " << ways << "\n";
    cout << "min: " << best << "\n";
    cout << "x y w\n";
}

```

```
for (auto& [x, y, w] : ans_figures) {  
    cout << x << " " << y << " " << w << "\n";  
}  
  
return 0;  
}
```